Python Mastery Assignment: Build Real Things

Nov 1, 2026 · @Bangash

This assignment turns every topic — from variables and loops up to testing — into small programs you would really use: shop bills, grade sheets, ATMs, libraries and chatbots. Each part shows where the idea is used in real life, then gives three tasks: Warm-up, Build and Challenge.

How this assignment works

Do the parts in order; each one only uses ideas from the parts before it. Every part has three levels, so every student can finish something and strong students still get stretched.

Level	Who it is for	Marks
Warm-up	Everyone — checks you understood the idea	2 per part
Build	A small real program using the idea	4 per part
Challenge	Extra thinking, optional for beginners	2 per part (bonus)
Final project	Everything together	30

Rules

1. One .py file per part, named like `part01_variables.py`, `part02_conditions.py`.
2. Start every file with a comment: your name, roll number and the date.
3. Your program must run without errors. Test it with at least two different inputs.
4. You may use AI to explain an error, but every line you submit must be one you can explain to the teacher. Expect to be asked “what does this line do?”
5. Submit all files in one folder (or a GitHub repo, using what you learned in the Git module).

How marks are given: works correctly (50%), clear names and comments (20%), handles wrong input (20%), neat output (10%).

Part 1 · Variables, data types, input and output

A variable is a labelled box that stores a value. Every app you use is full of them.

Where you see this in real life

- A shop bill stores the item name (str), quantity (int) and price (float).
- Your student ID card: name, roll number, class, blood group.
- A mobile top-up screen reads your number and amount with input, then prints a receipt.

Warm-up — Student ID card

Ask for name, age, class and city, then print a neat card. Also print the type of each value with `type()`.

```
======== STUDENT CARD ========
Name : Ali Raza
Age  : 17
Class: 1st Year
City : Wazirabad
==============================
```

Build — Shop bill

Ask for one item's name, price and quantity. Print the subtotal, 5% discount, 17% tax on the discounted amount, and the final total, each rounded to 2 decimals.

Challenge — Unit converter

Ask for a temperature in Celsius and a distance in kilometres. Print Fahrenheit (`C × 9/5 + 32`), Kelvin, miles (`km × 0.621371`) and metres, using f-strings with 1 decimal place.

> Remember: `input()` always gives text. Convert with `int()` or `float()` before doing maths.

Part 2 · Conditions (if / elif / else)

Conditions let a program make decisions: "if this is true, do that, otherwise do something else."

Where you see this in real life

- Result cards: marks decide the grade (A, B, C, Fail).
- Cinema or bus tickets: children and seniors pay less.
- Login screens: correct password → enter, wrong → error message.

Warm-up — Even, odd and sign

Ask for a number. Print whether it is positive, negative or zero, and whether it is even or odd.

Build — Grade calculator

Ask for marks out of 100. Print the grade using this table, and print “Invalid marks” for anything below 0 or above 100.

Marks	Grade
80–100	A
70–79	B
60–69	C
33–59	D
0–32	Fail

Challenge — Electricity bill

Ask for units used. The first 100 units cost Rs 10 each, the next 100 cost Rs 15, and every unit above 200 costs Rs 20. Add Rs 50 fixed charges. Print the total. Example: 250 units → 1000 + 1500 + 1000 + 50 = Rs 3550.

> Test the edges: 0, 100, 101, 200 and 201 units. Bugs hide at boundaries.

Part 3 · Loops (for / while)

Loops repeat work so you don’t copy the same line 100 times. Use `for` when you know how many times; use `while` when you repeat until something happens.

Where you see this in real life

- An ATM gives you 3 tries to enter the right PIN (while loop).
- A teacher adds up 40 students’ marks (for loop).
- A game keeps running until you press Quit (while True + break).

Warm-up — Times table

Ask for a number and print its table from 1 to 10 in the form `7 x 3 = 21`.

Build — ATM PIN check

The correct PIN is 4321. Give the user 3 tries. On success print “Welcome”; after 3 wrong tries print “Card blocked”. Show tries left after each mistake.

Challenge — Number guessing game

Use `random.randint(1, 50)` to pick a secret number. Keep asking for guesses, saying "Too high" or "Too low", and count the guesses. When correct, print "Got it in 6 guesses!" Then ask "Play again? (y/n)".

> Extra practice: print a triangle of stars with nested loops, and find the sum of all even numbers from 1 to 100 (answer: 2550).

Part 4 · Lists, dictionaries, tuples and sets (Module 1)

Collections hold many values in one variable. Pick the one that matches the job: a list for an ordered group, a dict to look things up by name, a tuple for fixed data, a set for unique items.

Where you see this in real life

- Phone contacts: name → number (dict).
- A playlist or a shopping list you can reorder (list).
- GPS location (latitude, longitude) that never changes (tuple).
- Counting unique visitors to a website (set).

Warm-up — Shopping list

Start with `items = ["milk", "bread", "eggs"]`. Add "tea", remove "bread", sort the list, and print each item with its number using `enumerate`.

Build — Phone book

Use a dict to store at least 5 contacts. Make a menu in a loop: 1) add, 2) search, 3) delete, 4) show all, 5) exit. Searching for a missing name must print "Not found", not crash (use `.get()` or `in`).

Challenge — Class result sheet

Store marks as `{"Ali": [80, 72, 91], "Sara": [95, 88, 90], ...}` for at least 5 students. Print each student's total and average, the class topper, and a set of all students who scored below 40 in any subject.

Part 5 · Functions and comprehensions (Module 2)

A function is a named, reusable block of code: write once, call many times. A comprehension builds a list or dict in one readable line.

Where you see this in real life

- A calculator app: every button calls a function like `add()` or `percent()`.
- An online shop runs the same `apply_discount()` for every product.
- Filtering a list of products to show only those under Rs 1000 (comprehension).

Warm-up — Small helpers

Write `area_of_circle(r)`, `is_even(n)` and `greet(name, lang="en")` where lang can be "en" or "ur" ("Salaam"). Call each one three times.

Build — Marks toolkit

Write `average(marks)`, `grade(avg)` (reuse your Part 2 table) and `report(name, marks)` that uses the other two and returns a one-line result string. Use them on 3 students.

Challenge — Comprehensions

In one line each: the squares of 1–10; only the names longer than 4 letters from a list; a dict of `{name: average}` from your Part 4 result sheet; and the passing students only (`average >= 33`).

Part 6 · References and mutability (Module 3)

Two names can point to the same list. Change it through one name and the other sees the change too. Knowing this prevents some of the most confusing bugs.

Where you see this in real life

- A shared Google Sheet: everyone with the link sees the same edits (same object).
- "Save a copy" before editing so the original stays safe (`.copy()`).
- A shopping cart that accidentally shows another customer's items (the shared default-list bug).

Warm-up — Predict, then run

Write your prediction as a comment first, then run the code and explain any surprise.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)          # prediction?

c = a.copy()
c.append(5)
print(a, c)       # prediction?
print(a is b, a is c, a == a[:])
```

Build — Fix the cart bug

This function shares one cart between all customers. Show the bug with two calls, then fix it using `None`.

```
def add_item(item, cart=[]):
    cart.append(item)
    return cart
```

Challenge — Safe class copy

Make `classes = {"A": ["Ali", "Sara"]}`. Make a shallow copy and a deep copy (`import copy`). Add a student to class A through each copy and print all three to show the difference.

Part 7 · Exceptions and files (Module 4)

Exceptions let a program handle problems politely instead of crashing. Files let it remember data after it closes.

Where you see this in real life

- A form says “Please enter a number” instead of crashing when you type letters.
- A diary or notes app saves every entry to a file.
- School results exported to a CSV file that opens in Excel.

Warm-up — Safe division

Ask for two numbers and divide them. Catch `ValueError` (not a number) and `ZeroDivisionError` with clear messages, and keep asking until the input is valid.

Build — Daily diary

Each run asks for today’s note and appends it to `diary.txt` with the date (`datetime.date.today()`). Add a menu option that reads and prints all past entries. If the file doesn’t exist yet, print “No entries yet”.

Challenge — CSV result sheet

Save your Part 4 result sheet to `results.csv` (name, three marks, average) using the `csv` module. Then write a second program that reads it back and prints the topper. Also save the same data as `results.json`.

Part 8 · Modules, packages and projects (Module 5)

Big programs are split into files (modules) and folders (packages). You also reuse other people’s code from the standard library and pip.

Where you see this in real life

- Every app on your phone is hundreds of files working together.
- Developers install ready-made tools with `pip` instead of rewriting them.
- AI apps keep their secret API key in a `.env` file, never in the code.

Warm-up — Standard library tour

Use `math` (square root of 144), `random` (pick a random student from a list), `datetime` (days left until 31 December) and `statistics` (mean and median of 5 marks).

Build — Your own module

Move your Part 5 functions into `school_tools.py`. Import them into `main.py` and use them. Add an `if __name__ == "__main__":` test block to `school_tools.py`, and show it does not run when imported.

Challenge — Real project setup

Create a virtual environment, install `requests`, save `requirements.txt`, and arrange this folder layout:

```
school_app/
├── school/
│   ├── __init__.py
│   ├── grades.py
│   └── attendance.py
├── main.py
├── requirements.txt
└── .gitignore
```

Part 9 · OOP — classes and objects (Module 6)

A class is a blueprint; an object is one real thing made from it, with its own data (attributes) and actions (methods).

Where you see this in real life

- A banking app: every account is an object with a balance and deposit / withdraw methods.
- A game: every player, enemy and bullet is an object.
- A library system: Book, Member and Library classes working together.

Warm-up — Student class

Make a `Student` class with name, roll number and a marks list. Add `add_mark()`, `average()` and a `__str__` that prints `Ali (Roll 12): avg 81.0`.

Build — Bank account

Make `BankAccount` with owner and balance. `deposit()` and `withdraw()` must refuse wrong amounts with `ValueError` (negative amount, or more than the balance). Keep a transaction history list and print it.

Challenge — Library system

Create `Book` (title, author, available) and `Library` that *has* many books (composition). Add `add_book()`, `borrow(title)`, `return_book(title)` and `available_books()`. Then make `EBook(Book)` that adds a file size — inheritance — and overrides `__str__`.

Part 10 · Iterators, generators and decorators (Module 7)

Generators hand out values one at a time with `yield`, saving memory. Decorators wrap a function to add extra behaviour without changing its code.

Where you see this in real life

- AI chat apps stream the answer word by word (a generator).
- Reading a huge log file one line at a time instead of loading it all.
- Websites use decorators like `@app.route("/")` and `@login_required`.

Warm-up — Countdown generator

Write `countdown(n)` with `yield` and loop over it. Then write `evens(limit)` and print `list(evens(10))` → `[0, 2, 4, 6, 8, 10]`.

Build — Big file reader

Create a text file with at least 50 lines, some containing the word ERROR. Write a generator `read_lines(path)` and a second generator `only_errors(lines)`. Count the error lines without ever building a list.

Challenge — Your own decorators

Write `@timer` (prints how long a function took) and `@log_calls` (prints the function name and its arguments). Use `*args, **kwargs`, `@wraps` and remember to return the result. Put both on a function that sums 1 to 1,000,000.

Part 11 · Type hints and dataclasses (Module 8)

Type hints label what kind of data each variable and function expects, so editors and mypy catch mistakes early. `@dataclass` writes the boring class code for you.

Where you see this in real life

- Big company codebases require hints so hundreds of developers understand each other's code.
- Product records in an online shop: name, price, stock (a dataclass).
- Checking an AI model's JSON answer with Pydantic before trusting it.

Warm-up — Add hints

Add type hints to all your Part 5 functions, including return types. Install mypy and run `mypy part05_functions.py`. Fix anything it reports.

Build — Product dataclass

Make `@dataclass Product` with name, price, stock and `tags: list[str]` (use `field(default_factory=list)`). Check in `__post_init__` that price and stock are not negative. Store 5 products in a list and print those that are out of stock.

Challenge — Save and load

Save your products to `products.json` using `asdict`, then load them back into `Product` objects with `Product(**d)`. Add `order=True` and sort the products by price.

Part 12 · Debugging and testing (Module 9)

Debugging finds and fixes bugs; tests check your code automatically so fixed bugs never come back.

Where you see this in real life

- Banking and hospital software runs thousands of tests before every update.
- Developers read logs to find why an app crashed for one customer.
- Tests prove that code written by an AI assistant really works.

Warm-up — Fix 3 bugs

This code should print the average of the passing marks (90 and 70), which is 80.0. Find and fix all three bugs using print debugging or `breakpoint()`, and write which kind of bug each one was.

```
marks = [90, 25, 70]
passing = []
for m in range(len(marks) - 1):
    if marks[m] > 33:
        passing = [marks[m]]
print(sum(passing) / len(marks))
```

Build — Test your grade function

Write `test_grades.py` with pytest. Use `@pytest.mark.parametrize` to test at least 8 marks, including every boundary (32, 33, 59, 60, 69, 70, 79, 80), and `pytest.raises` for marks above 100 (first change grade() so it raises ValueError for marks outside 0–100).

Challenge — Test the bank account

Use a `@pytest.fixture` that returns an account with Rs 1000. Test deposit, withdraw, the history list, and both errors. Run `pytest -v` and paste a screenshot of all tests passing.

Final project · School Manager (30 marks)

Build one menu-driven program that a real school office could use. It joins every part of this assignment into a single project.

```
===== SCHOOL MANAGER =====
1. Add student
2. Add marks
3. Show report card
4. Class ranking
5. Save and exit
Choose: _
```

What it must do

Feature	Topics it uses	Marks
Menu that keeps running until the user exits	Loops, conditions	3
`Student` dataclass with name, roll number and marks list	Dataclasses, type hints	4
Report card with average and grade	Functions, conditions	4
Class ranking, highest average first	Lists, sorting, lambda	3
Data saved to and loaded from `school.json`	Files, JSON	4
Wrong input never crashes the program	Exceptions	4
Code split into `school/` package + `main.py`	Modules, packages	3
At least 6 passing pytest tests	Testing	5

Rules: every function has type hints; no global variables except constants; one function does one job; use `logging` instead of debug prints.

AI bonus (+5)

Add a menu option “Write comment” that creates a short teacher comment from a student’s marks, for example “Excellent in all subjects, keep it up.” Write it as a `make_comment(student)` function with simple rules first. If you can use an AI model, put the API key in `.env`, stream the reply with a generator, retry with a decorator, and test it with a mocked model so the tests run offline.

Submit: your project folder or GitHub link, a screenshot of `pytest` passing, and a 5-line `README.md` explaining how to run it.